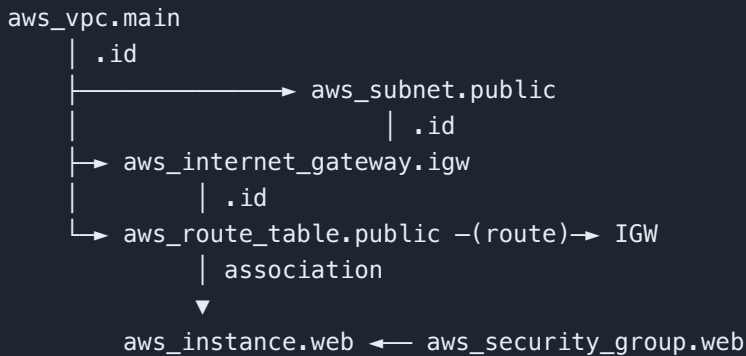


Week 2. 리소스 문법 & State (VPC → EC2)

개념 오늘의 목표 & 핵심 순간 ★

VPC / 서브넷 / IGW / 라우트 / SG / EC2를 코드로 엮는다. 핵심은 **암묵적 의존성(implicit dependency)**: 우리가 순서를 지정하지 않아도, 리소스가 서로의 속성(`aws_vpc.main.id` 등)을 참조하면 Terraform이 **의존성 그래프**를 만들어 알아서 순서대로 생성한다.



실습 1. provider + VPC + 서브넷

week2/main.tf :

```
terraform {
  required_version = ">= 1.9"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 6.0" }
  }
}

provider "aws" {
  region = "ap-northeast-2"
}

resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_hostnames = true
  tags = { Name = "boaz-w2-vpc" }
}

resource "aws_subnet" "public" {
  vpc_id            = aws_vpc.main.id          # ← 암묵적 의존성
  cidr_block        = "10.0.1.0/24"
  availability_zone = "ap-northeast-2a"
  map_public_ip_on_launch = true
  tags = { Name = "boaz-w2-public" }
}
```

실습 2. IGW + 라우트 테이블 + 연결

```
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id
  tags   = { Name = "boaz-w2-igw" }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
  tags = { Name = "boaz-w2-public-rt" }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}
```

실습 3. 보안 그룹 + EC2 (t3.micro, 프리티어)

```
data "aws_ami" "al2023" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name     = "name"
    values   = ["al2023-ami-*-x86_64"]
  }
}

resource "aws_security_group" "web" {
  name            = "boaz-w2-web-sg"
  description     = "allow ssh from my ip"
  vpc_id          = aws_vpc.main.id

  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"] # 실무선 내 IP로 제한. 실습 편의상 개방
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
  tags = { Name = "boaz-w2-web-sg" }
}

resource "aws_instance" "web" {
  ami                = data.aws_ami.al2023.id
  instance_type      = "t3.micro" # 프리티어
  subnet_id          = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.web.id]
  tags = { Name = "boaz-w2-web" }
}
```

실습 4. apply

```
terraform init
terraform plan # Plan: 7 to add 예상
terraform apply
```

확인 체크포인트

- Plan: 7 to add, 0 to change, 0 to destroy.

- EC2 콘솔에 `boaz-w2-web` 이 `running` , 퍼블릭 IP 할당됨

개념 ★ 의존성 그래프 눈으로 보기

우리는 어디에도 "VPC 먼저, EC2 나중"이라고 안 썼다. 참조(`aws_vpc.main.id`)가 곧 순서다.

```
terraform graph          # DOT 형식으로 의존 관계 출력
```

관찰 🛠 기록

- `terraform graph` 출력에서 `aws_instance.web` 이 의존하는 리소스: __
- `terraform state list` 의 리소스 개수(data 소스 제외): __

실습 5. state 구조 관찰

```
terraform state list
terraform state show aws_instance.web    # id, private_ip, subnet_id 등 확인
```

함정 InvalidAMIID / AZ 오류

- AMI 필터가 안 맞으면 `data.aws_ami` 가 빈 결과 → 리전이 `ap-northeast-2` 인지 확인.
- `Unsupported availability zone` : `ap-northeast-2a` 가 안 되면 `2c` 로 교체.

정리 destroy 체크리스트

```
terraform destroy    # Plan: 0 to add ... 7 to destroy
terraform state list # 비어있어야 함
```

- ☐ EC2 종료 확인(요금 방지) · VPC 삭제 확인
- ☐ `Destroy complete! Resources: 7 destroyed.`